Institut Supérieur des Études Technologiques de Djerba
Département Technologies de l'Informatique
Licence Appliquée — Développement des Systèmes d'Information

COMPTE RENDU DE TP

Application Android « Contacts » V2

Kotlin • Android Studio • Interfaces XML

Réalisé par : Bouaoun Meriem
Classe : L3DSI
Année universitaire : 2026 – 2027
Dépôt GitHub : github.com/meriembou216/TP_Android_Contacts

1. Introduction et objectifs

L'objectif de ce TP est de réaliser une application Android en Kotlin de gestion de contacts, composée de deux activités : `MainActivity`, qui affiche la liste des contacts, et `DetailActivity`, qui affiche et permet de modifier un contact.

Ce TP permet de mettre en pratique les notions suivantes :

- la **ListView personnalisée** avec un Adapter (`BaseAdapter`) ;
- la navigation entre activités avec un **Intent explicite** et le passage de données (`putExtra`) ;
- les **Intents implicites** (composeur téléphonique, application de messagerie, galerie, caméra) ;
- le **retour de résultat d'activité** avec l'API `registerForActivityResult` ;
- la gestion des **permissions** à l'exécution (caméra) et des cas d'erreur.

2. Structure du projet

Fichier	Rôle
`Contact.kt`	*Data class* qui représente un contact : nom, prénom, téléphone, e-mail et photo (`photoUri`, null si pas de photo).
`ContactAdapter.kt`	Adapter (hérite de `BaseAdapter`) qui remplit chaque ligne de la ListView à partir d'un objet `Contact`.
`MainActivity.kt`	Affiche la liste, ouvre `DetailActivity` au clic et reçoit les modifications.
`DetailActivity.kt`	Affiche et modifie le contact ; boutons Appeler, Envoyer un e-mail, Choisir / Prendre une photo, Enregistrer et Annuler.
`activity_main.xml`	Écran principal : barre de titre « Mes contacts » + `ListView`.
`item_contact.xml`	Modèle d'une ligne : photo, nom + prénom, numéro de téléphone.
`activity_detail.xml`	Écran de détail : photo, 4 champs `EditText` et les boutons d'action (dans un `ScrollView`).
`ic_person.xml`	Icône vectorielle affichée quand le contact n'a pas de photo.
`AndroidManifest.xml`	Déclaration des deux activités, de la permission `CAMERA` et de la caméra comme fonctionnalité non obligatoire.

3. Principales étapes de réalisation

3.1 Le modèle Contact

```
data class Contact(
    var nom: String, var prenom: String,
    var telephone: String, var email: String,
    var photoUri: String? = null
)
```

La photo est stockée sous forme d'adresse (Uri) en texte, ce qui facilite son passage d'une activité à l'autre dans un Intent.

3.2 ListView personnalisée

Le layout `item_contact.xml` définit le modèle d'une ligne. La classe `ContactAdapter` redéfinit `getView()` : elle réutilise la vue si elle existe (`convertView`), sinon elle la crée avec `LayoutInflater`, puis elle remplit la photo, le nom/prénom et le téléphone. Si `photoUri` est null, l'icône par défaut est affichée.

3.3 Ouverture de DetailActivity (Intent explicite)

Au clic sur une ligne, un Intent explicite désigne précisément l'activité à ouvrir. Les données du contact et sa position dans la liste sont ajoutées avec `putExtra`, à l'aide de clés (constantes) définies dans le `companion object` de `DetailActivity`.

```
val intent = Intent(this, DetailActivity::class.java)
intent.putExtra(DetailActivity.EXTRA_POSITION, position)
intent.putExtra(DetailActivity.EXTRA_NOM, contact.nom)
// ... prénom, téléphone, e-mail, photo
detailLauncher.launch(intent)
```

On utilise un lanceur (`registerForActivityResult` avec `StartActivityForResult`) au lieu de `startActivity`, car MainActivity doit recevoir une réponse.

3.4 Appeler et envoyer un e-mail (Intents implicites)

Un Intent implicite décrit l'action souhaitée et laisse Android choisir l'application capable de la réaliser :

```
Intent(Intent.ACTION_DIAL, Uri.parse("tel:$numero"))      // composeur
Intent(Intent.ACTION_SENDTO, Uri.parse("mailto:$adresse")) // messagerie
```

`ACTION_DIAL` ouvre le composeur sans lancer l'appel, donc aucune permission n'est nécessaire. `ACTION_SENDTO` avec `mailto:` ne propose que les applications de messagerie. Chaque `startActivity` est placé dans un `try / catch (ActivityNotFoundException)` pour afficher un message si aucune application compatible n'est installée.

3.5 Photo depuis la galerie et la caméra

Lanceur	Rôle	Résultat
`GetContent()` avec `"image/*"`	Ouvre la galerie pour choisir une image	Uri de l'image, ou null si annulé
`TakePicturePreview()`	Ouvre l'appareil photo	Bitmap, ou null si annulé
`RequestPermission()`	Demande la permission `CAMERA`	true / false

Dans les deux cas, l'image est **copiée dans le dossier privé de l'application** (`filesDir`) et c'est l'adresse de cette copie qui est conservée. Ainsi, MainActivity peut toujours afficher la photo, même sans les droits d'accès temporaires accordés par la galerie.

Avant d'ouvrir la caméra, l'application vérifie que l'appareil en possède une (`FEATURE_CAMERA_ANY`), puis vérifie ou demande la permission.

3.6 Enregistrer / Annuler (retour de résultat)

Le bouton **Enregistrer** vérifie d'abord les champs. S'ils sont valides, il place les données modifiées dans un nouvel Intent, appelle `setResult(RESULT_OK, resultat)` puis `finish()`. Le bouton **Annuler** appelle `setResult(RESULT_CANCELED)` puis `finish()`.

Dans MainActivity, le lanceur reçoit le résultat. Si le code vaut `RESULT_OK`, le contact est mis à jour à sa position, puis `adapter.notifyDataSetChanged()` redessine immédiatement la liste avec les nouvelles informations et la nouvelle photo.

```
if (result.resultCode == RESULT_OK && data != null) {
    val position = data.getIntExtra(DetailActivity.EXTRA_POSITION, -1)
    if (position in contacts.indices) {
        contacts[position].nom = data.getStringExtra(DetailActivity.EXTRA_NOM) ?: ...
        // ... autres champs
        adapter.notifyDataSetChanged()
    }
}
```

3.7 Gestion des situations courantes

Situation	Traitement
Champ obligatoire vide	Message d'erreur sur le champ (`EditText.error`) et l'écran reste ouvert ; vérification du format de l'e-mail avec `Patterns.EMAIL_ADDRESS` .
Annulation du choix de photo	Le lanceur renvoie null : message « Aucune photo choisie » / « Photo annulée », la photo précédente est conservée.
Aucune application compatible	`try / catch (ActivityNotFoundException)` : message explicite au lieu d'un plantage.
Permission caméra refusée	Message « Permission caméra refusée ».
Caméra indisponible	Test de `FEATURE_CAMERA_ANY` : message « Aucune caméra disponible ».

4. Tests effectués

Les tests ont été réalisés sur l'émulateur **Pixel 8** d'Android Studio.

#	Test	Résultat attendu / obtenu	État
1	Lancement de l'application	La liste des 5 contacts s'affiche (photo, nom, prénom, téléphone).	**OK**
2	Clic sur un contact	DetailActivity s'ouvre avec les champs préremplis.	**OK**
3	Modification + Enregistrer	Le nom (Dupont → Dupon) et la photo sont mis à jour immédiatement dans la liste.	**OK**
4	Bouton Appeler	Le composeur s'ouvre avec le numéro du contact.	**OK**
5	Bouton Envoyer un e-mail	L'application Gmail s'ouvre.	**OK**
6	Choisir une photo	La galerie s'ouvre ; l'image choisie s'affiche dans DetailActivity.	**OK**
7	Prendre une photo	Demande de permission, ouverture de la caméra, photo affichée.	**OK**
8	Annuler une opération photo	Message affiché, l'ancienne photo est conservée.	**OK**
9	Champ Nom vide + Enregistrer	Erreur « Nom obligatoire », rien n'est enregistré.	**OK**
10	Modification puis Annuler	Message « Modifications annulées », la liste reste inchangée.	**OK**

Captures d'écran

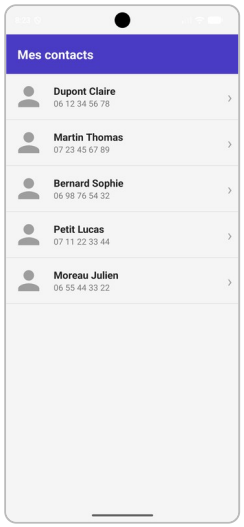


Fig. 1 — MainActivity : liste des contacts

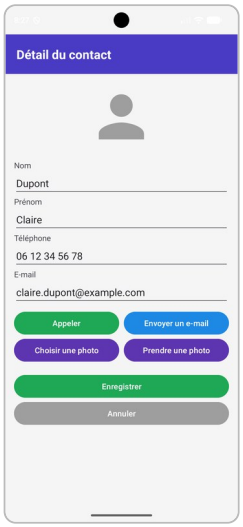


Fig. 2 — DetailActivity

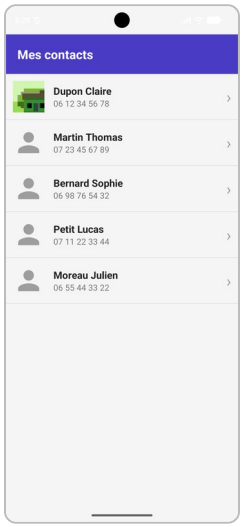


Fig. 3 — Liste mise à jour après Enregistrer

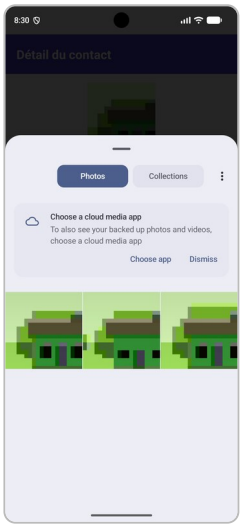


Fig. 4 — Choix d'une photo dans la galerie

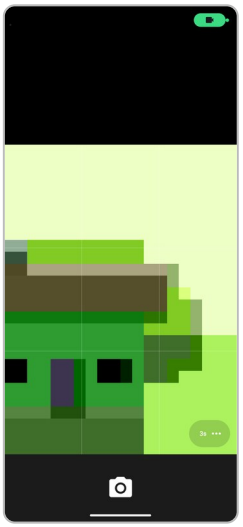


Fig. 5 — Prise de photo avec la caméra

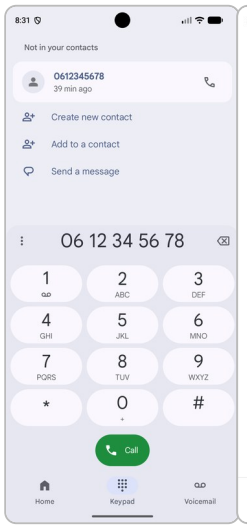


Fig. 6 — Composeur avec le numéro

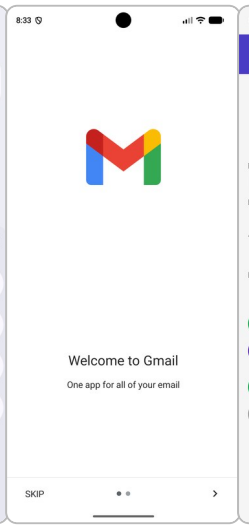


Fig. 7 — Ouverture de l'application de messagerie

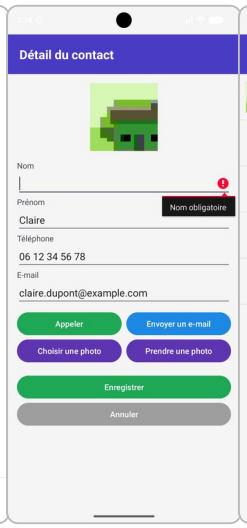


Fig. 8 — Champ obligatoire vide

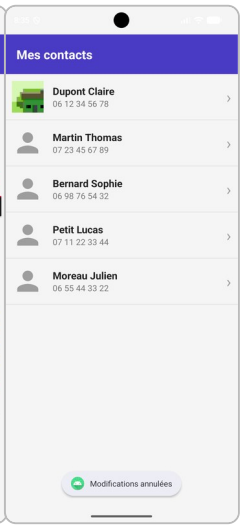


Fig. 9 — Annulation des modifications

5. Difficultés rencontrées

Difficulté	Cause et solution
Erreurs *Unresolved reference* sur `EXTRA_POSITION`, `EXTRA_NOM`…	MainActivity utilisait les clés avant que le `companion object` soit ajouté dans `DetailActivity` (le fichier contenait encore le code généré). Solution : compléter DetailActivity avec ses constantes.
Titre « Mes contacts » caché sous la barre d'état	Les versions récentes d'Android affichent l'application bord à bord. Solution : ajouter `android:fitsSystemWindows="true"` sur le layout racine.
Trois packages portant le même nom	Android Studio crée aussi des packages *(test)* et *(androidTest)* pour les tests automatiques. Les classes du TP ont été placées dans le package principal.
Galerie vide sur l'émulateur	Ajout d'images par glisser-déposer d'un fichier depuis le PC sur l'émulateur.
Accès à la photo de la galerie depuis MainActivity	L'image choisie est copiée dans le dossier privé de l'application pour garder un accès permanent.
Dossier `captures` absent du dépôt GitHub	Le dossier a été créé après le premier partage ; il a été ajouté ensuite par un nouveau commit.

6. Conclusion

Ce TP m'a permis de réaliser une application Android complète à deux activités. J'ai compris la différence entre un **Intent explicite**, qui désigne une activité précise de l'application, et un **Intent implicite**, qui décrit une action à réaliser par une autre application. J'ai aussi appris à faire un aller-retour de données entre activités avec `registerForActivityResult`, à personnaliser une ListView avec un Adapter et à gérer les permissions et les cas d'erreur pour que l'application ne plante pas.

Comme amélioration possible, on pourrait ajouter l'ajout et la suppression de contacts, ainsi que la sauvegarde des contacts dans une base de données (Room) pour les retrouver après la fermeture de l'application.